

SMART STUDENT MANAGEMENT SYSTEM

Project Goal

Design and build a Smart Student Record Analyzer using only functions and file handling in Python.

The program should not only store and manage student records but also analyze and summarize data such as average marks, top performers, and grade insights.

Students must think logically and design their own functions, data handling, and validation approaches.

Learning Objectives

Use functions to structure logic (no classes or OOP).

Design program flow and data management logically.

Practice file I/O for persistent data storage.

Perform basic data analysis using built-in Python logic.

Handle user errors, validation, and edge cases effectively.

Suggested Functional Structure

Students must plan their code into modular functions.

Example layout:

```
def add_student():
def view_students():
def search_student():
def update_student():
def delete_student():
def analyze_data():
def validate_input():
def main():
```

Students must decide which functions call others and how to share data between them.

Data Storage

All student records must be saved in a text file named students.txt in the format:

id,name,age,grade,marks

Example:

101,Ali Ahmed,17,A,92

102,Sana Khan,18,B,76

Functional Requirements

Add Student:

Ask for ID, Name, Age, Grade, and Marks with input validation; append to file.

View Students:

Display all data neatly and show total number of records.

Search Student:

Allow searching by ID or partial name (case-insensitive search recommended).

Update Student:

Allow selective updates for any field and rewrite file.

Delete Student:

Confirm before deleting and rewrite the file.

Analyze Data:

Implement at least three analytical functions such as:

Average marks

Top performer

Below-average count

Highest / lowest marks

Exit:

Gracefully end the program with a thank-you message.

Input Validation Rules

ID must be an integer and unique.

Name must not be empty or numeric.

Age must be an integer greater than 0.

Grade must be a single uppercase letter (A–F).

Marks must be an integer between 0 and 100.

Invalid input should trigger a clear message and re-prompt.

Analytical Thinking Challenges

Design your own function flow and file rewrite logic.

Handle invalid or missing data in file intelligently.

Avoid code repetition by using helper functions.

Implement at least two custom analytical insights of your own design.

Write meaningful comments explaining logic and structure.

Sample Console Flow

===== Smart Student Record Analyzer =====

1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Analyze Data
7. Exit

Enter your choice: 6

--- Data Analysis ---

Average Marks: 78.5

Top Performer: Ali Ahmed (92)

Students Below Average: 3

Highest Marks: 95 | Lowest: 52

—